ELEC6234
Embedded Processor Design

FPGA synthesis of a microprocessor core
Notes on ALU synthesis

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Approaches to ALU synthesis

- How to synthesise ADD and SUB
  - ADD is synthesised using A+B
  - picoRISC does not detect a Carry signal on the most significant bit, however a simple extension can provide that functionality
- Subtraction can be implemented using 2's complement subtraction rule:
  - A – B = A + ~B + 1
  - This way the same hardware can be used for ADD and SUB
- 2's complement overflow is not detected but can be handled using a simple extension

Writing synthesisable SystemVerilog code for combinational logic synthesis

- when writing code for decoders or multiplexers, use conditional statements
  - if-then-else
  - case

- common trap:

  conditional statements with incompletely specified signals cannot be mapped into combinational logic:

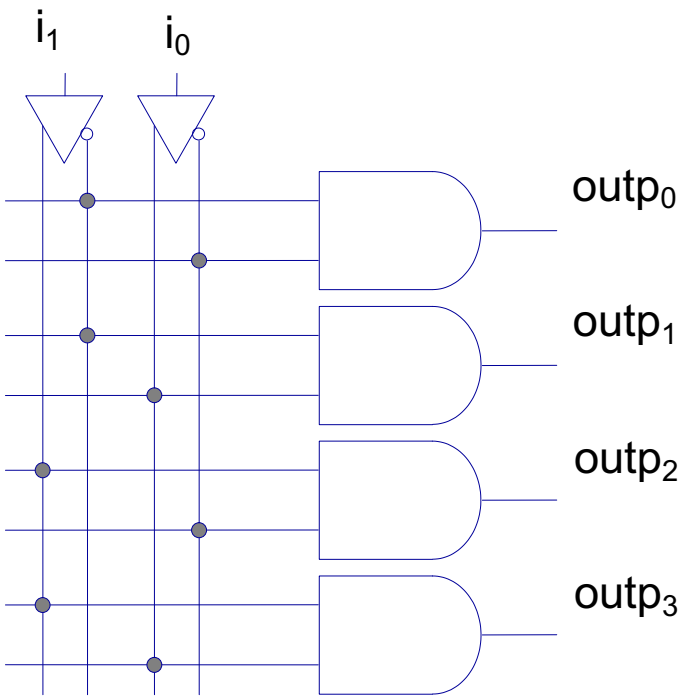
```
if(a)
   b = 1'b0; // b is incompletely specified
```

- use the **default** clause
  - assign default values to all signals driven by combinational logic blocks (examples will follow)

- in arithmetic circuit synthesis
  - arithmetic operations + - are mapped into standard adders/subtractors
  - Synthesis tools, e.g. Synplify Pro/Quartus map the multiplication operator * into dedicated multipliers if available, or combinational (cellular) multipliers – fast but very costly !

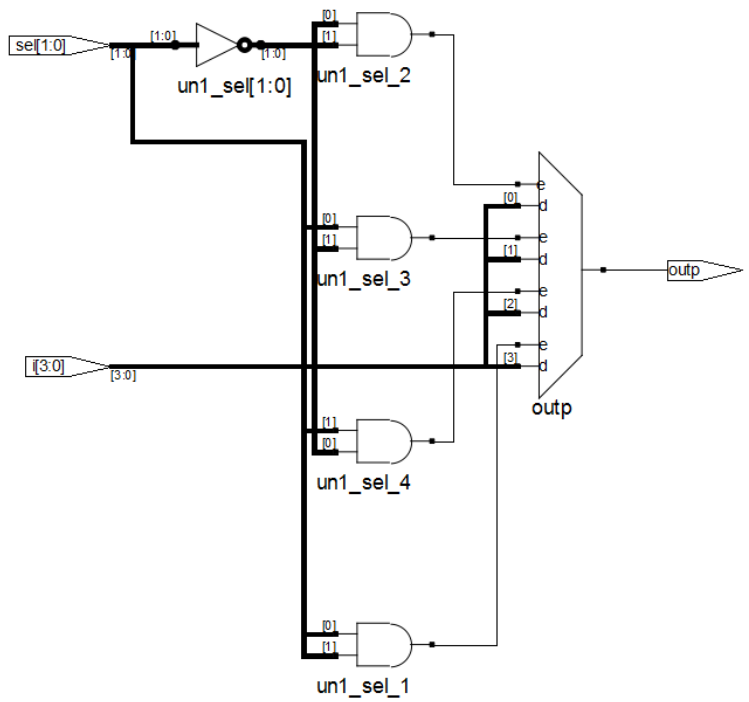
Decoder example

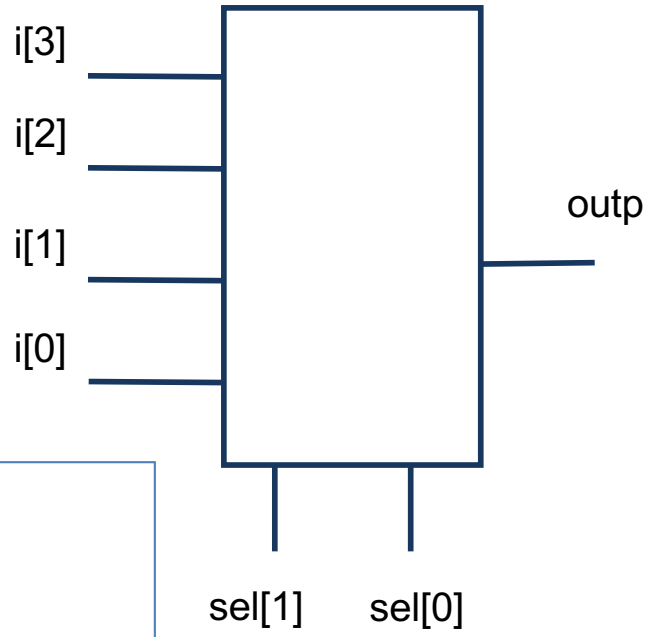
```
module decoder2to4(
      input logic [1:0] i, output logic [3:0] outp);

always_comb // note absence of sensitivity clause
begin
  outp = 4'b0000; // default value
  unique case (i) // unique will force completeness test
     0: outp[0] =  1'b1;
     1: outp[1] =  1'b1;
     2: outp[2] =  1'b1;
     3: outp[3] =  1'b1;
   endcase
end

endmodule
```



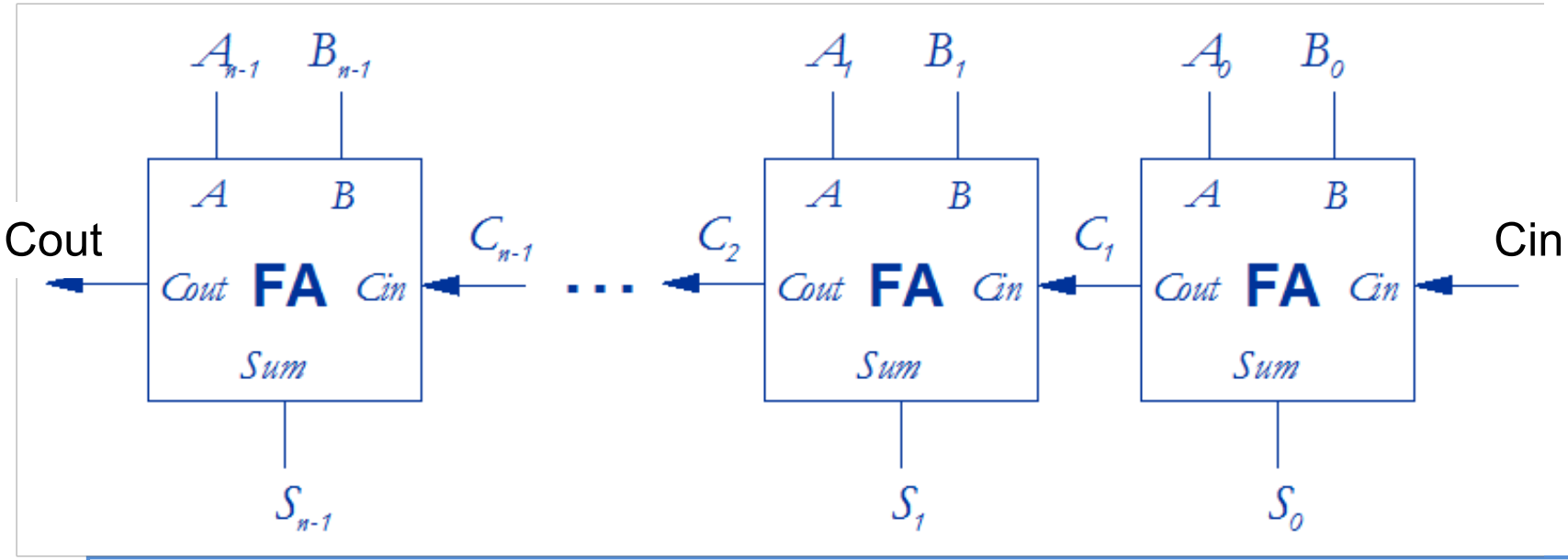
Multiplexer example

```
module mux4to1(
      input logic [3:0] i, input logic [1:0] sel, output logic outp);

always_comb // note absence of sensitivity clause
  unique case (sel) // unique will force completeness test
     0: outp =  i[0];
     1: outp =  i[1];
     2: outp =  i[2];
     3: outp =  i[3];
   endcase


endmodule
```



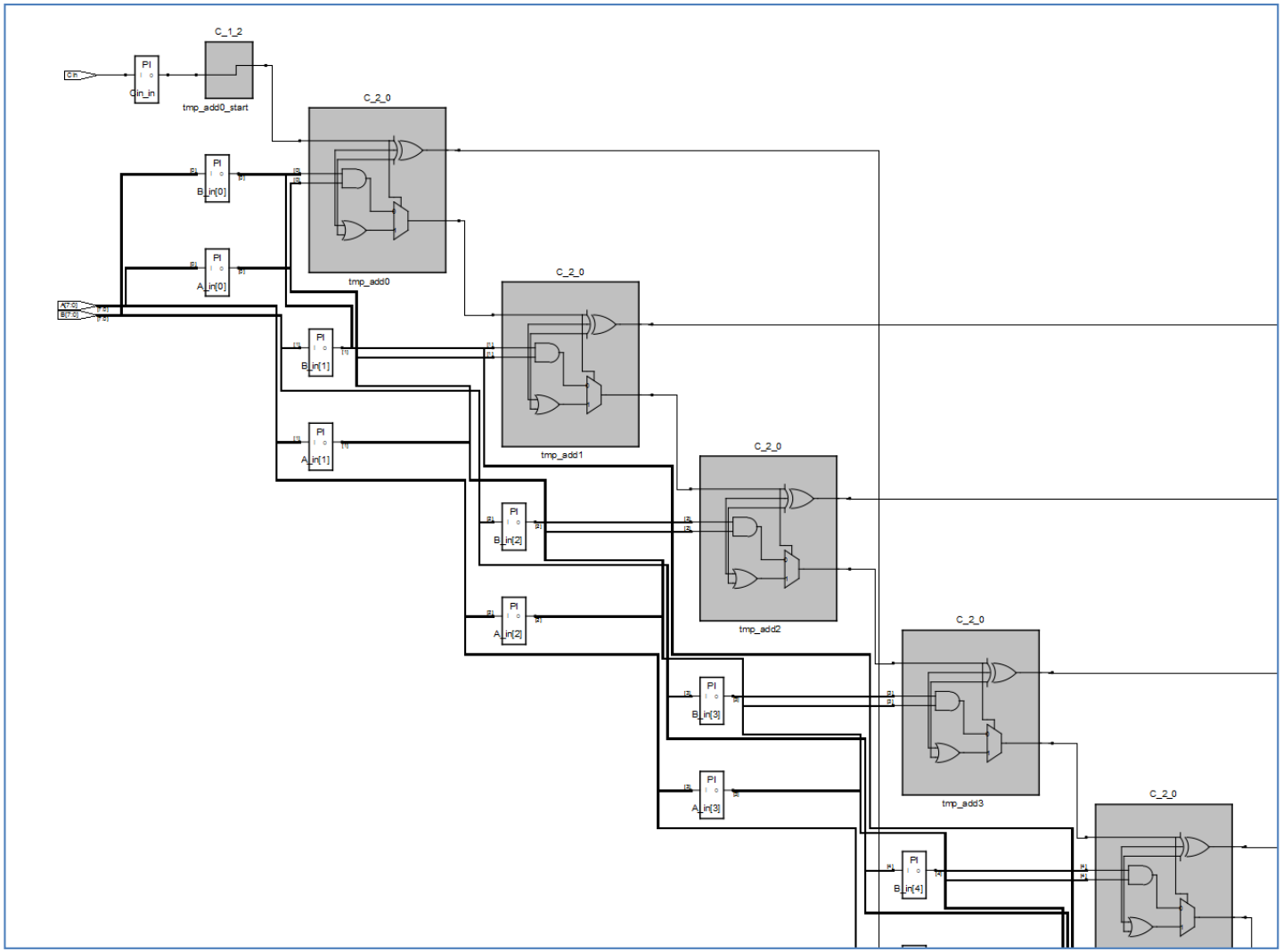


Arithmetic circuits - 8-bit adder

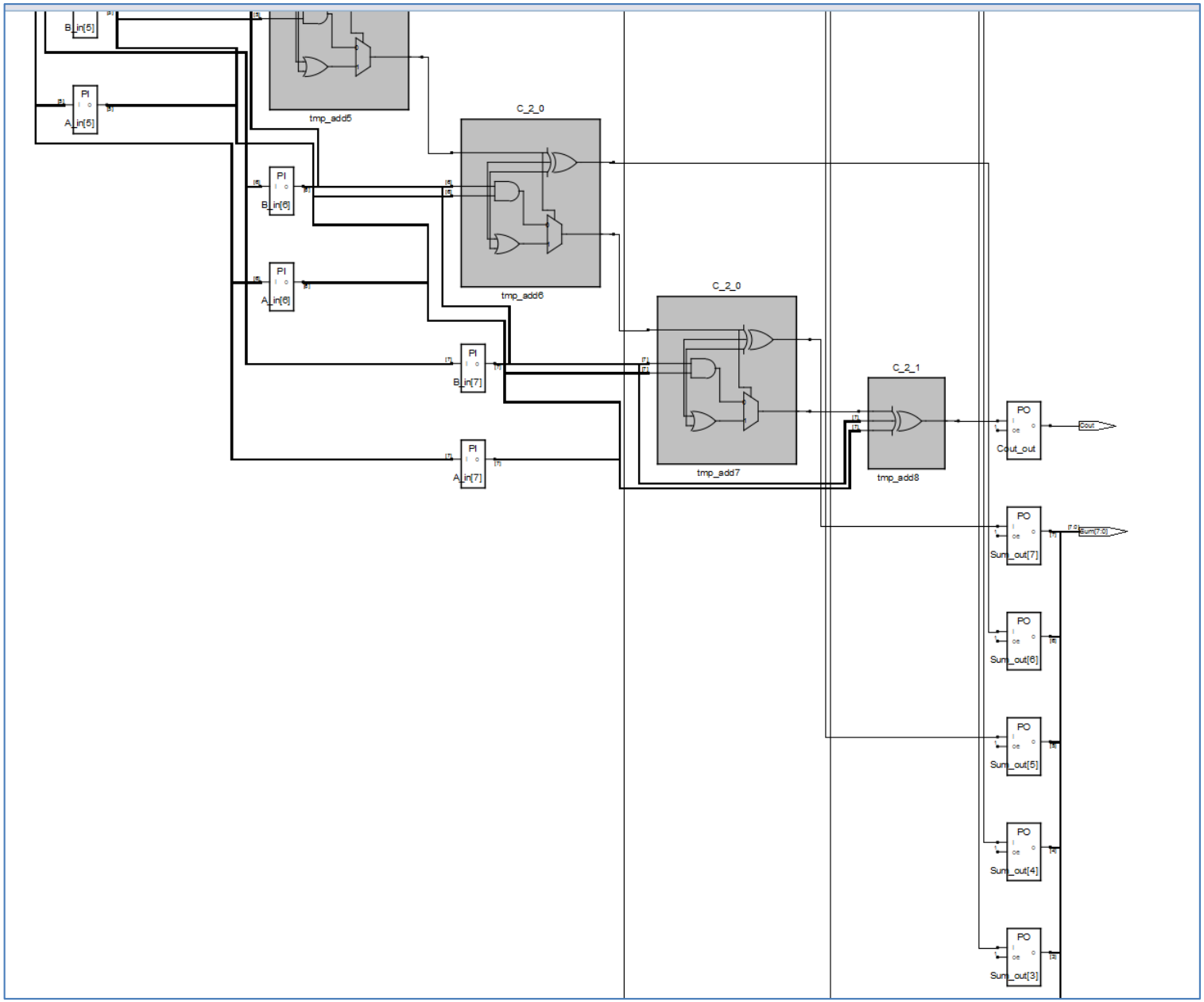



```
// 8-bit adder -  if Cout is needed, it can be obtained simply by extending the addition to 9 bits
module adder8
   (input logic [7:0] A, B, input logic Cin,
    output logic [7:0] Sum, output logic Cout);

  assign {Cout,Sum} = {1'b0,A} + {1'b0,B} + Cin; // extend addition by 1 bit

endmodule
```

Arithmetic circuits - 8-bit adder synthesised for Altera Cyclone III FPGA (1)



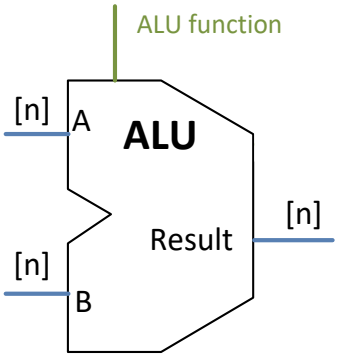
Arithmetic circuits - 8-bit adder synthesised for Altera Cyclone III FPGA (2)



Sample ALU module

```
//-----------------------------------------------------
// File Name   : alu.sv
// Function    : ALU module for picoRISC
// Version: 1,  only 8 funcs
// Author:  tjk
// Last rev. 20 Feb 23
//-----------------------------------------------------

`include "alucodes.sv"
module alu #(parameter n =8) (
   input logic [n-1:0] a, b, // ALU operands
   input logic [2:0] func, // ALU function code
   output logic [n-1:0] result // ALU result
);
…
```

```
//-----------------------------------------------------
// File Name   : alucodes.sv
// Function    : pMIPS ALU function code definitions
// Author:   tjk
// Last rev. 23 Oct 12
//-----------------------------------------------------
//
`define RA   3'b000
`define RB   3'b001
`define RADD  3'b010
`define RSUB  3'b011
`define RAND  3'b100
`define ROR   3'b101
`define RXOR  3'b110
`define RNOR  3'b111
```



ALU module - cont

```
…
// create the ALU, use signal ar in arithmetic operations
// create an n-bit adder and build the ALU around the adder
logic[n-1:0] ar,b1; // temp signals
always_comb
begin
  if(func==`RSUB)
    b1 = ~b + 1'b1; // 2's complement subtrahend
  else b1 = b;
  ar = a+b1; // n-bit adder
end // always_comb

always_comb
begin
result = a; // default
 case(func)
   `RA   : result = a;
   `RB   : result = b;
   `RADD  : result = ar;
   `RSUB  : result = ar;
   `RAND  : result = a & b;
  `ROR   : result= a | b;
  `RXOR  : result = a ^ b;
  `RNOR  : result = ~ (a | b);
  endcase
end //always_comb

endmodule //end of module ALU
```

The first always_comb block creates an adder and the logic for –B needed in subtraction.

The adder result (ar) is used in a separate always_comb block.

alu.sv

Branch conditions can be implemented in a separate module, outside of the ALU

LT = 1 if Rd-Rs < 0 in 2's complement arithmetic (i.e. sign bit =1)
EQ = 1 if Rd-Rs = 0 i.e. Rd and Rs are the same
LTU = 1 if Rd < Rs in unsigned arithmetic (ALU would generate C=1 for Rd-Rs

Branch comparator is faster than ALU

Implementation of branch comparator

```
//-----------------------------------------------------
// File Name   : branch.sv
// Function    : Branch comparator module
// Author:  tjk
// Last rev. 14 Feb 25
//-----------------------------------------------------

module branch #(parameter n =8) (
   input logic [n-1:0] a, b, // input operands
   output logic LT,EQ, LTU // comparison result
);
// define signed versions of input operands as logic is unsigned by default
logic signed as,bs;
assign as = signed'a;
assign bs = signed'b;

assign LT = (as<bs? 1'b1:1'b0);
assign EQ =(a==b? 1'b1:1'b0);
assign LTU = (a<b? 1'b1:1'b0);

endmodule;
```

RISC-V branch instructions can be implemented easily using EQ,LT and LTU

- BEQ %rd,%rs, imm; if EQ==1, PC += imm
- BNE %rd,%rs, imm; if EQ==0, PC += imm
- BGE %rd,%rs, imm; if LT == 0, PC += imm
- BGEU %rd,%rs, imm; if LTU == 0, PC += imm
- BLT %rd,%rs, imm; if LT == 1, PC += imm
- BLTU %rd,%rs, imm; if LTU == 1, PC += imm

Unconditional branches:

- JAL %rd, imm; %rd <= PC+4, PC += imm
- JALR %rd, imm(%rs); %rd <= PC+4, PC := %rs+imm

JALR and subroutine calls

- Call a subroutine (near call, within the imm range)
  - CALL offset -> JALR %rd, imm(%0); %rd <= PC+4, PC := imm

- Return from subroutine
  - RET -> JALR %0, imm(%0); PC <= %rd

Summary

- A simple ALU can be developed without condition codes, but they can be added if necessary; will be discussed next
- Branch condition can be handled by a separate block which uses comparators and is faster than the ALU
- Next we will discuss how branches can be implemented